

SyDent — مرجع نظام المحاسبة الشامل

الإصدار: 1.2 — 05 يونيو 2026 **يقابل commit:** ab6efea على main (سياق v77). الإصدار 1.1 كان caf722 ، والإصدار 1.0 كان 1b824
6b. **الصفحات:** accounting.html (المحاسبة على مستوى العيادة) + patient-profile.html (دفتر حساب المريض) + patients.htm
1 (سجل المرضى) + index.html (لوحة التحكم — حالة الدفع للحظية) **الفلسفة:** Cash Basis Accounting (محاسبة نقدية) — مطابق
OpenDental + Dental CFO standards

< **تحديث الإصدار 1.2 (05 يونيو 2026):** يوثق إصلاحات جلسة مراجعة المحاسبة (سياق v70 وما بعده). ثلاثة تغييرات جوهرية: < 1.
تحسين الإيراد لحظة القراءة — القاعدة الموحدة صارت مطبقة داخل accounting.html مباشرةً عبر splitIsEarned() +
(loadSplitSessionStatuses()) ، فالملخص صحيح بالبناء بمعزل عن self-heal ملف المريض. (كانت في 1.1 «تقوية اختيارية مستقبلية».) <
2. **ميزة التسويات (Migration 45)** — جدول account_adjustments بثلاثة أنواع (خصم / إعدام / استرداد)، مدموجة في طبقتي العيادة
والمريض. القسم 15 الجديد يشرحها بالكامل. (سيناريو الاسترداد في القسم 6 ما عاد نظرياً.) < 3. **مزامنة لوحة التحكم** — index.html
صارت تحسب حالة الدفع لحظياً من الدفتر (computeDashFinancials) بنفس القاعدة الموحدة، بدل الحقل المخزن patients.payment. <
< **تحديث الإصدار 1.1 (28 مايو 2026):** أضيفت القاعدة الموحدة للجلسات المخططة (انظر القسم 14): المال المستلم قبل
اكتمال العلاج يبقى **رصيذاً مقدماً (unearned)** ولا يصير إيراداً إلا بعد اكتمال الجلسة. الرصيد على مستوى المريض يُحسب على **العلاج
المنجز فقط**. هذا التحديث صَحَّ خلافاً كان يُخصّص الدفعات للجلسات المخططة (إيراد بلا إنتاج). كل التفاصيل السابقة محفوظة كما هي.

جدول المحتويات

1. [الفلسفة المحاسبية](#)
2. [مصادر البيانات الستة](#)
3. [المعادلات الأساسية \(7\)](#)
4. [نظام الـ Buckets للحفاظ على Revenue Conservation](#)
5. [المسارات الكاملة \(Data Flow\)](#)
6. [السيناريوهات والـ Edge Cases](#)
7. [TIMESTAMP TZ Timezone Safety](#)
8. [Anti-Double-Counting](#)
9. [Identity Tests الـ 5](#)
10. [Defense in Depth \(Owner-only\)](#)
11. [Render Pipeline](#)
12. [Common Bugs & Troubleshooting](#)
13. [Code Layout Map](#)
14. [نموذج حساب المريض + قاعدة الجلسات المخططة \(v1.1\)](#)
15. [التسويات: خصم / إعدام / استرداد \(v1.2 — Migration 45\)](#)

1. الفلسفة المحاسبية

Cash Basis (محاسبة نقدية)

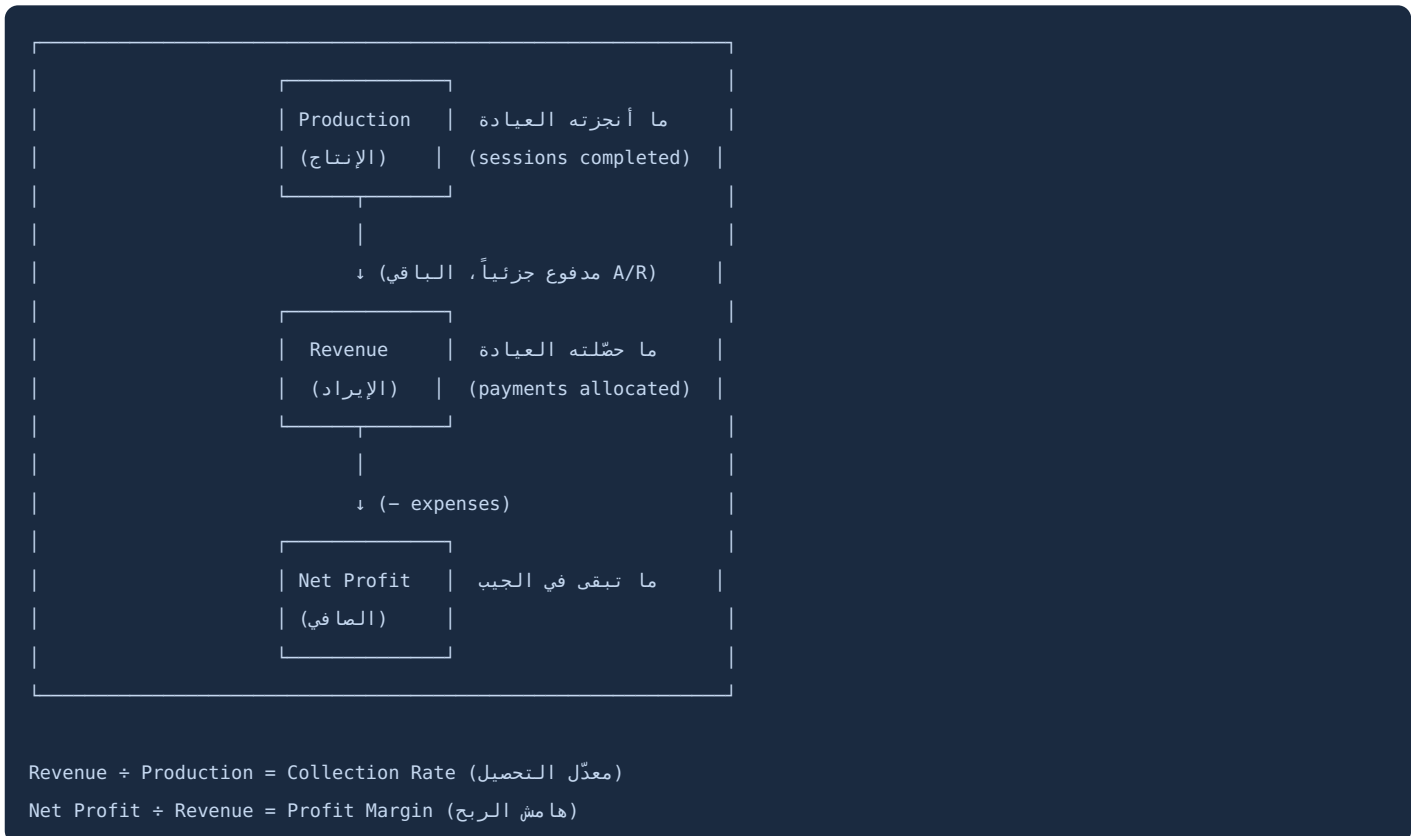
< **القاعدة الذهبية:** الإيراد يُحسب عند **استلام المال**، لا عند **اكتساب الإنتاج**. والمصروف يُحسب عند **دفع المال** (أو إصدار الالتزام للمخبر).

لماذا Cash Basis وليس Accrual؟

المعيار	Cash Basis	Accrual
متى يُحسب الإيراد	عند استلام المال من المريض	عند إكمال الجلسة (حتى لو لم يُدفع)
متى يُحسب المصروف	عند الدفع	عند نشوء الالتزام
التعقيد	بسيط، مباشر	

المعيار	Cash Basis	Accrual
		accounts receivable/payable معقّد، يتطلب tracking
مناسب للسوق السوري	نعم — أغلب العيادات تتعامل cash	مناسب أكثر للعيادات مع insurance
OpenDental default		يحتاج setup إضافي
Dental CFO recommendation	Many dental practices use cash-basis "accounting"	ل corporate-level reporting فقط

المفاهيم الثلاثة الأساسية



Unearned Income (الدفعات المقدّمة)

< Critical: Unearned credits ليست إيرادات محاسبياً.

```

100 (deposit) قبل أن يحدث أي علاج للمريض دفع
↓
100 payment + split بـ is_unearned=true
↓
liability (التزام على العيادة) - revenue هي - هذه ليست
↓
accounting page: section تظهر في (yellow) منفصل
لا تدخل في صافي الربح أو الإيراد المعروض
  
```

ال **accrual rule**: عندما الجلسة تكتمل وال FIFO ينقل ال 100k من unearned إلى session-allocated، حينها تصبح revenue.

< v1.1 — القاعدة الموحّدة (Canonical Rule): ال split يُعتبر إيراداً مكتسباً (allocated) فقط إذا تحقّق الشرطان معاً:

is_unearned = false و ال split مرتبط بجلسة مكتملة (completed). أي split على جلسة مخطّطة (planned) — حتى لو

is_unearned=false — يُعامل كـ رصيد مقدّم (unearned). هذا يطابق وضع "OpenDental" Rigorous حيث دفعات الجلسات المخطّطة

تبقى prepayment حتى اكتمال الإجراء. التفاصيل الكاملة في القسم 14.

2. مصادر البيانات الستة (+ التسويات)

الجدول الشامل

#	الجدول	الـ Column Owner	حقول التاريخ	نوع الحقول	الاستخدام
1	ledger_sessions	doctor_id (legacy)	date	DATE	الإنتاج المنجز
2	ledger_payments	doctor_id (legacy)	date	DATE	المدفوعات المستلمة (raw)
3	payment_splits	doctor_id (legacy)	(via payment_id)	—	تخصيص المدفوعات للجلسات والأطباء
4	expenses	owner_id	date	DATE	المصاريف التشغيلية
5	provider_payouts	owner_id	paid_at	TIMESTAMPZ	رواتب وحصص الأطباء / الموظفين
6	lab_orders	doctor_id (legacy)	date_sent	TIMESTAMPZ	تكاليف المخابر
7	account_adjustments	doctor_id (legacy)	date	DATE	التسويات: خصم / إعدام / استرداد — Migration 45) (القسم 15)

Reference Data (للـ render فقط)

#	الجدول	الـ Column Owner	الاستخدام
7	clinicDoctors	owner_id	للحصول على اسم/لون/active كل طبيب
8	expenseCategories	owner_id	للحصول على اسم/أيقونة/لون كل تصنيف مصاريف
9	clinicEmployees	owner_id	loaded but not used in current Phase 7.5 logic — kept for (future

مصادر البيانات في الكود

```
// In accounting.html state vars:
var doctors = []; // clinicDoctors
var expenseCategories = []; // expenseCategories
var allEmployees = []; // clinicEmployees (reserved for future)
var sessions = []; // ledger_sessions (in-range, status=completed)
var payments = []; // ledger_payments (in-range)
var paymentSplits = []; // payment_splits (for in-range payments)
var expenses = []; // expenses (in-range)
var payouts = []; // provider_payouts (in-range, timezone-trimmed)
var labOrders = []; // lab_orders (in-range, !rejected, timezone-trimmed)
var adjustments = []; // account_adjustments (Migration 45): discount / write_off / refund
var splitSessionStatus = {}; // id->status يشير إليها split لكل جلسة (لتطبيق القاعدة الموحدة لحظة القراءة)
```

< v1.2: المتغيران adjustments و splitSessionStatus أُضيفا في جلسة مراجعة المحاسبة. splitSessionStatus يُجلب عبر loadSplitSessionStatuses() (انظر القسم 3 / المعادلة 1 + القسم 15).

3. المعادلات الأساسية (7)

Revenue: Equation 1 (الإيرادات المحصلة)

```
splitsRevenue =  $\sum$  paymentSplits.amount حيث is_unearned = false
unearnedTotal =  $\sum$  paymentSplits.amount حيث is_unearned = true
splitPaymentIds = Set من كل payment_id في paymentSplits

legacyRevenue =  $\sum$  ledger_payments.amount حيث p.id NOT IN splitPaymentIds

revenue = splitsRevenue + legacyRevenue
```

شرح: - splitsRevenue = ما تم تخصيصه من المدفوعات لـ sessions محددة (cash earned). - unearnedTotal = ما تم استلامه لكن لم يُخصَّص بعد (cash received but NOT earned). - legacyRevenue = مدفوعات قديمة (قبل Migration 3) بدون splits — نأخذ amount كاملاً.

Anti-double-counting: لو payment له split → يُحسب فقط في splitsRevenue. لو ليس له split → يُحسب في legacyRevenue.

< v1.2 — القاعدة الموحدة مطبقة لحظة القراءة (splitIsEarned): صار accounting.html يطبق القاعدة الموحدة مباشرةً في مسار حساب الإيراد، لا يكفي بقراءة is_unearned. لكل split يُسأل splitIsEarned(sp) < < patient- < < js > > // مطابق > > . كان على جلسة مكتملة + unearned إيراد مكتسب فقط إذا لم يكن split = computeFinancials: > > // profile.html > > function splitIsEarned(sp) { > > if (!sp || sp.is_unearned) return false; > > return !!sp.session_id && > > splitSessionStatus[sp.session_id] === 'completed'; > > } > > (سلوك FIFO قديم) يُحسب الآن كـ unearned لا كإيراد. حالات splitsRevenue < < : splitIsEarned(sp) === true → splitsRevenue → loadSplitSessionStatuses() يجلب كل جلسة يشير إليها split (chunked بـ 150، مستقل عن مصفوفة sessions لأن جلسة الـ split قد تكون مكتملة-خارج-الفترة أو مخططة — وكلتاها قد لا تكون في sessions). جلسة legacy بحالة '' / null تُطوى إلى complete d. عند خطأ في الجلب، تبقى الجلسة غير-مخططة في الماب → splitIsEarned يعاملها كـ غير مكتملة (محافظ: يتجنب نفخ الإيراد). < < **النتيجة:** splitsRevenue صحيح بالبناء الآن، بمعزل عن أي كاتب أنتج splits نظيفة أو عمّا إذا كان self-heal ملف المريض قد جرى. (في 1.1 كان هذا «تقوية اختيارية مستقبلية»، وقد نُقِّد).

Production: Equation 2 (الإنتاج)

```
production =  $\sum$  ledger_sessions.cost حيث:
doctor_id = currentUser.id
AND status = 'completed'
AND date BETWEEN [from, to]
```

شرح: سعر كل الجلسات المكتملة في الفترة. planned sessions **مستثناة** (لأنها لم تتم بعد) — مطابق Total Prod "OpenDental".definition

Collection Rate: Equation 3 (معدل التحصيل)

```
collectionRate = (production > 0)
? (revenue / production) * 100
: null // الـ "-" في UI
```

Benchmark: أكثر من 90% = ممتاز. أقل من 70% = مشكلة A/R كبيرة.

Total Expenses: Equation 4 (المصروفات الإجمالية)

```
expensesTotal =  $\sum$  expenses.amount (in-range)
payoutsTotal =  $\sum$  provider_payouts.amount (in-range, timezone-trimmed)
labsTotal =  $\sum$  lab_orders.cost (in-range, !rejected, timezone-trimmed)
```

```
totalExpenses = expensesTotal + payoutsTotal + labsTotal
```

شرح: المصاريف مقسومة لـ 3 buckets منفصلين لا يتداخلون: - **expenses**: مصاريف تشغيلية (إيجار، كهرباء، صيانة...) — لا تحتوي رواتب - **payouts**: رواتب وحصص الأطباء والموظفين - **labs**: تكاليف المخابر (incurred at date_sent)

Equation 5: Net Profit (صافي الربح)

```
netProfit = revenue - totalExpenses

// Display logic:
if (netProfit >= 0) → " صافي الربح (green)
else → " صافي الخسارة (red, show absolute value)
```

Equation 6: Profit Margin (هامش الربح)

```
profitMargin = (revenue > 0)
                ? (netProfit / revenue) * 100
                : null // "-" في UI

// سالب profitMargin → سالب netProfit و revenue > 0
// revenue=100k, expenses=250k → margin = -150% مثلاً
```

Equation 7: Per-Provider Performance

```
(active أو deactivated) D لكل طبيب:
production_D = Σ sessions.cost حيث provider_id = D.id

labCost_D = Σ matching_labs.cost (de-duped via sharedMatchedSet)
            session.date من ±90 نفس patient_id + نفس tooth_num
            matching =

collection_D = Σ paymentSplits.amount حيث provider_id = D.id AND splitIsEarned(sp)
                + Σ (legacy_payment[patient] × provider_ratio[patient])
                حيث provider_ratio = provider's_production_for_patient / total_production_for_patient

paidOut_D = Σ provider_payouts.amount حيث provider_id = D.id

clinicNet_D = collection_D - labCost_D - paidOut_D
```

شرح clinicNet: - إذا موجبة → الطبيب جلب للعيادة أكثر مما أخذ منها (الطبيب profitable للعيادة). - إذا سالبة → الطبيب أخذ أكثر مما حَصَلَت العيادة من إنتاجه (طبيعي عند الـ pre-payments للطبيب).

4. نظام الـ Buckets للحفاظ على Revenue Conservation

المبدأ

كل ل.س في revenue يجب أن يُحسب في bucket واحد بالضبط في providerRows. هذا يضمن:

```
Σ providerRows[].collection = revenue (Test A)
```

Bucket 1: Known Doctors (الأطباء النشطون أو المعطلون لكن موجودون)

```
// sessions/splits/payouts) وله نشاط في الفترة clinic_doctors كل طبيب موجود في
doctors.forEach(function(doc){
  if (!providerIdsInData[doc.id]) return; // skip empty doctors
  rows.push(computeOneProviderRow(doc, ...));
});
```

ال collection: - splits ب legacy proportional - provider_id = doc.id (non-uneared) للمرضى الذين لديهم sessions on this doctor

Bucket 2: _unassigned (غير محدد)

```
// يُضاف لو:
// - sessions ب provider_id=null في الفترة، أو
// - splits ب provider_id=null + !is_uneared، أو
// - orphan legacy payments (في الفترة sessions مرضى بدون)
if (hasUnassignedSessions || hasUnassignedSplits || orphanLegacyRevenue > 0) {
  rows.push(unassignedRow);
}
```

ال collection: - splits ب legacy proportional - provider_id=null (non-uneared) للجلسات بدون provider + orphanLegacyRevenue (المرضى الذين دفعوا لكن لا توجد لهم sessions in-range)

Bucket 3: _deleted_doctors (أطباء محذوفون)

```
// provider_id ل sessions/splits/payouts references يُضاف لو في
// doctors table الحالي (FK SET NULL after deletion) لا يوجد في ال
Object.keys(providerIdsInData).forEach(function(pid){
  if (knownDoctorIds[pid]) return; // already covered
  // ... aggregate orphanProduction, orphanCollection, etc.
});
```

ال collection: - splits ب legacy proportional - provider_id = deleted_id (non-uneared) للجلسات على هذا الطبيب المحذوف - payouts المسجلة على هذا ال id

Bucket 4 (Implicit): non-clinical payouts

(...سكرتيرة، سائق) employee_id-only في الجدول. هي مدفوعات row هذه ليست totals.paymentsTotal (وبالتالي في) totalExpenses تحسب في clinical لأنها ليست - per-provider table لكن لا تظهر في nonClinicalPayouts > 0 إذا footnote تُظهر مذكرة في ال

Conservation Proof

```
Σ providerRows.collection
= Σ (known doctors)
+ (_unassigned bucket: splits + legacy + orphanLegacy)
+ (_deleted_doctors bucket: splits + legacy + payouts)

= ALL splits non-uneared (each split has provider_id one of: known/null/orphan)
+ Σ proportional legacy for patients with in-range sessions
  (ratios across all doctors sum to 1.0 per patient ⇒ each patient's full legacy is allocated)
+ Σ orphanLegacyRevenue (patients with no in-range sessions ⇒ all in _unassigned)
```

```
= splitsRevenue + legacyRevenue
= revenue ✓
```

5. المسارات الكاملة (Data Flow)

مسار 1: صفحة تُفتح

1. CDN loads: window.supabase exists
2. supabase-init.js runs: creates window.sb, window.sbGetUser, exposes window.SyDentLock
3. sidebar.js runs: defines initSidebar
4. DOM finished parsing
5. Inline <script> runs: init() called

```
init():
  ↓
  initSidebar('accounting') ← inject sidebar HTML
  ↓
  window.sbGetUser()         ← get auth user
  ↓
  guardPage(['owner'])       ← block non-owners
  ↓
  Promise.all([              ← parallel load reference data
    loadDoctors(),
    loadExpenseCategories(),
    loadEmployees()
  ])
  ↓
  set default range (current month)
  ↓
  wire event listeners
  ↓
  logAudit('report.view_accounting') ← audit log entry
  ↓
  applyFilter()              ← first data load
```

مسار 2: المستخدم يضغط "تطبيق" أو quick-range button

```
applyFilter():
  ↓
  check _applyInFlight guard
  ↓
  read fromDate / toDate values
  ↓
  validate (from <= to, both non-empty)
  ↓
  set _applyInFlight = true, disable btnApply
  ↓
  show loading state
  ↓
  loadAllInRange(from, to):
    ↓
```

```

Promise.all([
    ← 6 parallel queries ( +loadAdjustments)
    loadSessions(),
    loadPayments(),
    loadExpenses(),
    loadPayouts(), ← timezone-trimmed
    loadLabOrders(), ← timezone-trimmed
    loadAdjustments() ← account_adjustments (DATE, Migration 45)
])
↓
loadPaymentSplits() ← sequential (filtered by payment_id)
    ↳ loadSplitSessionStatuses() ← جلب كل جلسة يشير إليها status
↓
render(from, to):
    ↓
    computeSummary() ← Equations 1, 2, 3 ( عبر splitIsEarned)
    ↓
    computeExpenseTotals() ← Equation 4
    ↓
    computeExpenseBreakdown() ← Equation 6 (per-category percentages)
    ↓
    computeProviderPerformance() ← Equation 7 (all 4 buckets, عبر splitIsEarned)
    ↓
    computeAdjustmentTotals() ← خصم/إعدام/استرداد (القسم 15)
    ↓
    حساب الأرقام الصافية: ←
        netRevenue = grossRevenue - refundsTotal
        netProduction = grossProduction - chargeReductions
        netProfit = netRevenue - totalExpenses
        collectionRate = grossRevenue ÷ netProduction
    ↓
    runIdentityTests() ← (لا تتأثر بالتسويات - GROSS) على الأرقام الإجمالية
    ↓
    build HTML:
        renderSummaryCards() ← 6 cards («مع سطور «صافي بعد الخصم/الاسترداد»)
        renderExpenseSection() ← breakdown with bars
        renderAdjustmentsSection() ← بطاقة التسويات (تظهر فقط عند وجودها)
        renderProviderSection() ← per-provider table
        renderUnearnedSection() ← optional yellow card
    ↓
    $('body').innerHTML = html
    ↓
    setTimeout: animate bar-fills
    ↓
    refreshDebugPanel() if open
↓
reset _applyInFlight = false, re-enable btnApply

```

مسار 3: Debug panel toggle (Ctrl+Shift+D)

keydown event:

```

↓
if (ctrl + shift + D):
    e.preventDefault()

```

```
toggleDebugPanel():
  ↓
  $('debugPanel').classList.toggle('show')
  ↓
  if shown:
    refreshDebugPanel():
      ↓
      render lastIdentities array
      ↓
      show ✓ green or x red for each test
```

6. السيناريوهات والـ Edge Cases

السيناريو 1: عيادة جديدة — لا data

```
sessions=[], payments=[], paymentSplits=[], expenses=[], payouts=[], labOrders=[]

summary = {
  revenue: 0,
  production: 0,
  collectionRate: null,    ← display "-"
  unearnedTotal: 0
}

totals = { totalExpenses: 0, ... }
breakdown = [] → empty state
providerRows = [] → empty state
netProfit = 0, profitMargin = null

✓ Render: empty states, صفر errors
```

السيناريو 2: شهر طبيعي — كل شيء يعمل

```
sessions = 50 sessions in May, production = 5M
payments = 30 payments, splits cover them all, splitsRevenue = 4M
unearned = 0
expenses = 500k (rent + electricity)
payouts = 1.5M (doctor salaries)
labs = 600k (date_sent in May)

summary.revenue = 4M
totalExpenses = 500k + 1.5M + 600k = 2.6M
netProfit = 4M - 2.6M = 1.4M
profitMargin = 35%
collectionRate = 4M / 5M = 80%

Identity tests: ✓✓✓✓
```

السيناريو 3: شهر فيه prepayment كبير

```
مقدّمًا لتليسات المريض دفع 200k
sessions لم تُخصّص لـ (بعد 200k unearned splits)

splitsRevenue = 0
unearnedTotal = 200k

في accounting:
  الإيرادات: 0
  unearned section (yellow): 200,000 ل.س.

بعد income ليس prepayment: محاسبياً صحيح
```

السيناريو 4: طبيب معطل (is_active=false)

```
كان فعّالاً في يناير، تم تعطيله في فبراير X طبيب
في فترة 31/03-01:
- sessions (هو معطل) X: جديدة على sessions
- splits (لو دفعات تتم الآن لإنتاج قديم) X: قديمة على splits
- لكن في فترة 31/03-01 فقط ما تم في الفترة

providerIdsInData[X.id] يكون true قد (sessions/splits/payouts in-range)
doctors.forEach لأنه في الـ X يضم doctors.array
badge "معطل" يظهر باللون الأحمر"

واضح visual indicator + تظهر بياناته للشفافية ✓
```

السيناريو 5: طبيب محذوف (FK SET NULL)

```
Y للطبيب row clinic_doctors حذف Owner الـ
DB: بعد الحذف، في
- sessions.provider_id = NULL (FK SET NULL)
- splits.provider_id = NULL
- payouts.provider_id = NULL

(وليس مرجعاً صحيحاً للطبيب المحذوف) NULL يصبح provider_id الـ، SET NULL بعد - WAIT
_deleted_doctors ليس، bucket_unassigned هذه الحالة تذهب لـ

قديم UUID = ما زال provider_id يحدث فقط إذا الـ _deleted_doctors
(أو NO ACTION أو DEFAULT مثلاً) SET NULL ليس FK وهذا يحدث لو الـ
لن يُملأ غالباً _deleted_doctors في الواقع → SET NULL هو FK الـ SyDent ولكن في

FK policy. لو في يوم تغرّر defensive measure موجود كـ bucket الـ
```

السيناريو 6: Syria UTC+3 — Timezone edge

```
Owner (local Syria time) في 22/05/2026 02:00 الصباح
وقت الحفظ = الآن، paid_at = 22/05/2026 بـ payout يدخل
JS code بيني: paidAtIso = "2026-05-22T02:00:00+03:00"
DB stores it as TIMESTAMPTZ → UTC = "2026-05-21T23:00:00Z"

:يفلتر "الشهر الحالي" (22/05-01) Owner، ثم في 22/05 صباحاً
```

```
widenStartUtc("2026-05-01") = "2026-04-30T00:00:00.000Z"
widenEndUtc("2026-05-22") = "2026-05-23T23:59:59.999Z"
```

DB query: paid_at BETWEEN 04-30 AND 05-23 UTC → يجد row بـ UTC=05-21T23:00

inLocalRange("2026-05-21T23:00:00Z", "2026-05-01", "2026-05-22"):

- new Date(utc).getFullYear/Month/Date = 22/05/2026 (local time in Syria)
- localDay = "2026-05-22"
- "2026-05-01" <= "2026-05-22" <= "2026-05-22" ✓
- يُحفظ به ✓

✓ payout بشكل صحيح في local-date 22/05.

السيناريو 7: Salary expense double-counting

"اسمها" رواتب الأطباء expenses في category أنشأ Owner الـ
k فيها بـ expense 100 وأضاف

نفسها لنفس الطبيب k فيها 100 provider_payouts في نفس الفترة، الـ

نتيجة:

```
expensesTotal 100 k يحتوي الـ
payoutsTotal 100 k نفسها يحتوي الـ
totalExpenses = 200k (!مضاعف)
```

الـ accounting page:

- ✓ SALARY_HINT_RE باستخدام category يكتشف اسم
- ✓ row أصفر على الـ chip يعرض
- ✓ expense section أعلى الـ warning banner يعرض

يجب أن Owner الـ:

- "رواتب" category من expense يحذف الـ
- فقط لرواتب الموظفين/الأطباء payouts يستخدم

السيناريو 8: Manual allocation للـ payment

المريض P 3 sessions: A(100k), B(50k), C(80k) total 230k
ك دفع 60

FIFO 60 k الافتراضي: A.paid = 60k ، بالكامل → A

manual_allocation: فمّل Owner لكن الـ

30k → A, 20k → B, 10k → C (split-غير FIFO)

manual_allocation flag = true

accounting: في

```
paymentSplits 3 rows يحتوي
splitsRevenue 60 = k يجمعها كلها ✓
revenue نفس manual أو FIFO - لا فرق محاسبياً
```

السيناريو 9: Refund (المريض يسترد دفعة)

k ثم استرد 30 ، k المريض دفع 100

DB: في

```
payment1: amount = 100k
```

```
payment2: amount = -30k (negative)
```

يحمل الاثنان loadPayments

في computeSummary:

- payment2 له splits أم لا؟
- amount=-30k, is_unearned=false: split بـ لو له
- ✓ splitsRevenue += -30k → النتيجة 70 k
- splits ليس له payment2 لو
- ✓ legacyRevenue += -30k → النتيجة 70 k

بشكل صحيح revenue يقلل refund الـ ✓

> ** v1.2:** هناك مساران للاسترداد الآن:

> 1. **split** (الموضح أعلاه) - نادر، قد ينشأ من تعديل بيانات قديم؛ مدعوم رياضياً في الجمع

> 2. **غير** الاسترداد** (Migration 45) `account\_adjustments` تسوية استرداد رسمية** عبر
التفاصيل. (مريض) `computeFinancials` و (عيادة) `computeAdjustmentTotals` بمبلغ موجب، وتُعالج في `kind='refund'` يُسجل كصف في القسم 15** (في 1.1 كان هذا السيناريو نظرياً لعدم وجود ميزة استرداد بالواجهة).

السيناريو 10: Lab cost matching window

```
session.date = 2026-05-15
```

```
lab_order.date_sent = 2026-08-12 (يوم لاحق 89)
```

```
window = ±90 يوم → match ✓
```

```
session.date = 2026-05-15
```

```
lab_order.date_sent = 2026-08-14 (يوم لاحق 91)
```

```
window = ±90 يوم → NO match
```

labsTotal (لـ date_sent in range) يُحسب في lab الـ

per-provider labCost لكن لا يُحسب في

7.7 TIMESTAMPTZ Timezone Safety

المشكلة

TIMESTAMPTZ يخزن قيمة UTC + timezone offset.

PostgreSQL .gte/.lte يقارن مع الـ UTC value فقط.

وقت 22/05 02:00 local date بـ payout يدخل Syria (UTC+3) في Owner:

→ JS builds: "2026-05-22T02:00:00+03:00"

→ DB stores UTC: "2026-05-21T23:00:00Z"

:"يفلتر "22/05" → Owner 22/05 ثم

Naive query: .gte('paid_at', '2026-05-22') = >= 2026-05-22 00:00:00 UTC

الـ row stored as 2026-05-21T23:00 < 2026-05-22T00:00 → EXCLUDED

الحل (Phase 7.5)

نمط ثلاثي:

```
// day بـ ±1 UTC توسيع النافذة في 1.
```

```
function widenStartUtc(isoDay) {
```

```
  if (!isoDay) return isoDay;
```

```

var d = new Date(isoDay + 'T00:00:00Z');
d.setUTCDate(d.getUTCDate() - 1);
return d.toISOString(); // "2026-04-30T00:00:00.000Z" 01-05-2026" ٠١"
}

function widenEndUtc(isoDay) {
  if (!isoDay) return isoDay;
  var d = new Date(isoDay + 'T23:59:59.999Z');
  d.setUTCDate(d.getUTCDate() + 1);
  return d.toISOString(); // "2026-05-23T23:59:59.999Z" 22-05-2026" ٢٢"
}

// 2. Query ٠١ wide window
.gte('paid_at', widenStartUtc(from))
.lte('paid_at', widenEndUtc(to))

// 3. Strict local-date filter client-side
function inLocalRange(timestamptz, from, to) {
  if (!timestamptz) return false;
  var d = new Date(timestamptz);
  if (isNaN(d.getTime())) return false;
  var y = d.getFullYear();
  var m = String(d.getMonth() + 1).padStart(2, '0');
  var day = String(d.getDate()).padStart(2, '0');
  var localDay = y + '-' + m + '-' + day;
  return localDay >= from && localDay <= to;
}

// Usage:
payouts = res.data.filter(function(p){
  return inLocalRange(p.paid_at, from, to);
});

```

متى يُطبَّق

Treatment	Type	Column
widen + trim	TIMESTAMPTZ	provider_payouts.paid_at
widen + trim	TIMESTAMPTZ	lab_orders.date_sent
direct compare (no widen)	DATE	ledger_sessions.date
direct compare	DATE	ledger_payments.date
direct compare	DATE	expenses.date
not filtered (we use payment_id)	DATE (nullable)	payment_splits.payment_date

Anti-Double-Counting .8

Layer 1: Schema separation

- employee لا يستطيع ربط expense بـ clinic_employees or clinic_doctors table doesn't reference expenses
- provider_payouts is a separate table specifically for compensation
- expenses never in — lab_orders is a column in lab_orders.cost

Layer 2: Default categories don't include salaries

```
DEFAULT_CATEGORIES = [
  { name: 'مواد سنّية', ... },
  { name: 'أدوات ومعدّات', ... },
  { name: 'إيجار وخدمات', ... },
  { name: 'كهرباء وماء', ... },
  { name: 'صيانة', ... },
  { name: 'تنظيف وتعقيم', ... },
  { name: 'تسويق', ... },
  { name: 'مواصلات وصيانة', ... },
  { name: 'مصاريف عامة', ... }
];
// by design - لا يوجد "رواتب" أو "موظفين"
```

Layer 3: Salary-hint regex detection

```
var SALARY_HINT_RE = /ذ|ر|ا|ت|ب|أ|ج|ر|أ|ج|ر|أ|ج|ر|موظف|موظفين|طبيب|أطباء|أطباء|حصة|حصص|تعويض/;

// breakdown في expense category لكل
if (SALARY_HINT_RE.test(name)) {
  row.warnDup = true;
  // أصفر بجانب الاسم chip
  // section تحذير في أعلى الـ banner
}
```

Layer 4: Lab matching de-duplication

```
var sharedMatchedSet = {}; // shared across ALL providers

// per-provider labCost يُحسب مرة واحدة فقط في lab كل
labOrders.forEach(function(lo){
  if (sharedMatchedSet[lo.id]) return; // already matched to another provider
  // ... try match
  if (matched) sharedMatchedSet[lo.id] = true;
});
```

Test E (Identity)

$\sum \text{per-provider labCost} \leq \text{labsTotal}$

عندما inequality strict الـ

- lab orders بدون matching session (different patient/tooth, or out-of-window)
- provider lab في تلك الحالة لا يُضافون لأي
- labsTotal لكن يبقون في

Test A: Revenue Conservation

```
sumProvCollection = Σ providerRows[].collection
diff = |sumProvCollection - summary.revenue|
PASS if diff < 1
```

يفشل لو: نسينا bucket من ال buckets الأربعة. على سبيل المثال، orphan legacy payments بدون _unassigned bucket.

Test B: Production Conservation

```
sumProvProduction = Σ providerRows[].production
diff = |sumProvProduction - summary.production|
PASS if diff < 1
```

يفشل لو: sessions مع provider_id لا يطابق أي provider في ال rows. مثلاً، sessions على deleted-doctor بدون _deleted_doctors.bucket.

Test C: Expense Sum

```
sumC = expensesTotal + payoutsTotal + labsTotal
diff = |sumC - totalExpenses|
PASS if diff < 1
```

يفشل لو: حساب totalExpenses داخل المنطق لا يطابق ال sum من ال 3 streams. (في الحالة الحالية، هذا identity tautology — $\text{totalExpenses computed as sumC by definition}$).

Test D: Net Profit Identity

```
netCheck = summary.revenue - totals.totalExpenses
PASS = always true (by construction)
```

فقط للإظهار في ال debug panel — لا يمكن أن يفشل.

Test E: Lab De-duplication

```
sumProvLab = Σ providerRows[].labCost
PASS if sumProvLab <= labsTotal + tolerance
```

يفشل لو: sharedMatchedSet ضاع بين ال rows. (race condition?) في الحالة الحالية، ال Set مُشَارَك بشكل صحيح.

ال Tolerance

```
var TOLERANCE = 1; // ل.س
```

السبب: legacy proportional يستعمل floating-point ratios. تراكم rounding errors قد يصل لـ 0.5 ل.س لـ clinic بـ 100 + patient. $\text{Tolerance} < 1$ يعكس "كل ل.س صحيح".

Defense in Depth (Owner-only) .10

Layer 1: Sidebar BLOCKED

```
// in sidebar.js:
var BLOCKED = {
```

```

owner:    [],
doctor:   [..., 'accounting'],
secretary: [..., 'accounting']
};
// → Non-owners لا يرون الـ link أصلاً

```

Layer 2: guardPage

```

// in init():
if (window.SyDentLock && window.SyDentLock.guardPage) {
  if (!window.SyDentLock.guardPage(['owner'])) return;
}
// → Non-owners يحصلون على "blocked screen"

```

Layer 3: Manual role check

```

var role = window.SyDentLock.getRole();
if (role !== 'owner') {
  window.location.href = 'index.html';
  return;
}
// → Hard redirect لو SyDentLock فشل بطريقة ما

```

السيناريو

Doctor mode (locked):

1. sidebar: "المحاسبة" hidden ✓
2. guardPage يعرض blocked screen → مباشرة /accounting.html لو حاول
3. manual redirect to /index.html → فشل somehow guardPage لو

Render Pipeline .11

Order

```

$('body').innerHTML =
  renderSummaryCards()      ← 6 cards in grid
  + renderExpenseSection()  ← breakdown table
  + renderProviderSection() ← per-provider table
  + renderUnearnedSection() ← optional yellow card (if unearned > 0)

```

Animation

```

// Initially bar-fill width = 0%
setTimeout(function(){
  document.querySelectorAll('.bar-fill').forEach(function(el){
    var w = el.getAttribute('data-w');
    if (w) el.style.width = w + '%';
  });
}, 20);
// → CSS transition 0.3s animates the bars

```

Hidden by default. Toggled via keyboard shortcut.

Displays:

- ✓ A. Revenue conservation Σ providers: X vs total: Y ($\Delta=Z$)
- ✓ B. Production conservation ...
- ✓ C. Expense sum ...
- ✓ D. Net profit identity ...
- ✓ E. Lab dedup ...

If any fails → red x with details = indicates real bug

Common Bugs & Troubleshooting .12

"Bug Pattern 1: "window.sbGetUser is not a function

السبب: ال Supabase CDN يُحمّل بعد supabase-init.js.

الحل: تأكد أن ال 3 scripts بهذا الترتيب في `<head>`:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script src="supabase-init.js?v=..."></script>
<script src="sidebar.js?v=..."></script>
```

Bug Pattern 2: Revenue conservation fails (Test A)

السبب المحتمل: orphan legacy payment ضاع.

التشخيص: 1. اضغط Ctrl+Shift+D → اقرأ 2 Test A detail. لو $\Delta > 0$ → في legacy revenue مفقود 3. تحقق من patients مع payments بدون 4 sessions in-range. تأكد أن orphanLegacyRevenue يُحسب ويُضاف لـ unassigned_

Bug Pattern 3: Payouts/Labs لا تظهر رغم وجودها

السبب المحتمل: TIMESTAMPTZ timezone issue.

التشخيص: 1. تحقق من ال paid_at actual UTC value في Supabase 2. تأكد أن widenStartUtc/widenEndUtc تُطبّق 3. تأكد أن inLocalRange يطابق local date بشكل صحيح

Bug Pattern 4: Numbers seem doubled

السبب المحتمل: Salary expense duplicated في expenses + payouts.

التشخيص: 1. ابحث عن chip أصفر في 2 expense breakdown. تحقق من categories باسم "راتب/موظف/طبيب/حصة" 3. احذف expense من category صحيح، أو غيّر ال category name

Bug Pattern 5: Stale data on page (cache issue)

السبب: browser cache.

الحل: 1. Hard refresh (Ctrl+Shift+R or Ctrl+F5) 2. تحقق من cache-bust version :?v=20260601b30 (يُحدّث فليت-وايد عند كل تغيير 3 inline JS). لو ال devs غيّروا ال inline JS بدون bump version → السبب

Bug Pattern 6: "حدث خطأ" + console error

التشخيص: 1. افتح 2 Console (F12) DevTools. ابحث عن 3 red error. لو "script" + reference to undefined function TypeError" 4 load order. لو "5 DB migration missing" or "42P01". لو "Network error" → check Supabase status

Bug Pattern 7: Identity test fails after data edit

السبب: انتقل بيانات بين أعمدة أو حذف بدون trigger مناسب.

التشخيص: 1. اضغط Ctrl+Shift+D → اقرأ كل 2 tests. لو 3 revenue issue → Test A fails. لو 4 production issue → Test B fails. تحقق من DB integrity: هل splits referenced parent payment موجود؟ - هل sessions.cost لا NULL؟ - هل provider_id references صحيحة؟

Code Layout Map .13

< **v1.2:** أرقام الأسطر أدناه من بنية الإصدار 1.0 وصارت **تقريبية** (الملف نما إلى ~1830 سطر بعد إضافة loadSplit/ splitIsEarned SessionStatuses والتسويات). الدوال الجديدة في تجمعاتها المنطقية: loadSplitSessionStatuses + splitIsEarned بعد loadPayment Splits مباشرة؛ loadAdjustments ضمن مُحملات المصادر؛ computeAdjustmentTotals + renderAdjustmentsSection في طبقتي الحساب والعرض (القسم 15). استعمل grep -n 'function <name>' للموضع الدقيق.

Top of file (HTML)

```
Lines 1-20: DOCTYPE + meta tags + cache headers
Lines 21-310: <style> CSS variables, layout, components
Lines 312-319: <head> scripts (CDN → init → sidebar) critical order
Lines 320-340: Page header + filter bar
Lines 350-368: Toast + debug panel HTML
Lines 370: <script> opens
```

JS code structure

```
Lines 372-400: Module header comment (architecture)
Lines 401-410: Global state vars

Lines 414-466: Helpers (escapeHtml, fmtSY, fmtPct, todayISO, isoDate, showToast)
Lines 467-487: Migration detector + timezone helpers
    - isMigrationMissing
    - widenStartUtc / widenEndUtc
    - inLocalRange

Lines 517-595: init() function

Lines 597-630: Reference data loaders
    - loadDoctors
    - loadExpenseCategories
    - loadEmployees

Lines 634-682: Quick range button helpers
    - setMonthRange
    - setLastMonthRange
    - setQuarterRange
    - setYearRange
    - setActiveQuickButton / clearActiveQuickButton

Lines 684-718: applyFilter() with concurrency guard

Lines 720-735: loadAllInRange()

Lines 736-887: Per-source data loaders
    - loadSessions
    - loadPayments
```

- loadPaymentSplits
- loadExpenses
- loadPayouts (timezone-trimmed)
- loadLabOrders (timezone-trimmed)

Lines 891-940: computeSummary() ← Equations 1, 2, 3
 Lines 942-970: computeExpenseTotals() ← Equation 4
 Lines 973-1040: computeExpenseBreakdown() ← Equation 6
 Lines 1042-1208: computeProviderPerformance() ← Equation 7 (4 buckets)
 Lines 1210-1311: computeOneProviderRow()

 Lines 1313-1385: runIdentityTests() ← 5 tests

 Lines 1387-1413: render() ← main orchestrator
 Lines 1415-1473: renderSummaryCards()
 Lines 1475-1528: renderExpenseSection()
 Lines 1530-1599: renderProviderSection()
 Lines 1601-1614: renderUnearnedSection()

 Lines 1616-1632: Debug panel

- toggleDebugPanel
- refreshDebugPanel

 Line 1651: init() ← Boot call

ملاحظات للمطورين المستقبليين

1. لا تضيف DB writes إلى accounting.html — الصفحة read-only by design. أي mutation يجب أن يكون في صفحة مخصصة (expenses, payouts, إلخ).
2. لا تستعمل DATE filters على TIMESTAMPTZ columns — استخدم النمط الثلاثي (widen + trim).
3. عند إضافة data source جديد: - أضيف global state var - أضيف load function مع graceful fallback - أضيف Promise.all في loadAllInRange - أضيف totalExpenses (لو expense) - أضيف bucket في per-provider لو ينطبق - أضيف test جديد في runIdentityTests للحفاظ على conservation - حدّث هذا الـ reference document
4. عند تغيير DB schema: - راجع كل graceful fallbacks (isMigrationMissing) - اختبر مع كل من pre-migration و post-migration DBs - حدّث الـ SELECT field lists لتطابق الـ schema الجديد
5. عند إضافة category warning detection: - حدّث SALARY_HINT_RE regex - اختبر مع categories بأسماء عربية وإنجليزية مختلطة
6. عند تعديل identity tests: - الـ TOLERANCE حالياً = 1 ل.س - لو زاد عدد المرضى لـ +1000، قد تحتاج زيادة tolerance لـ 5-10 ل.س - لا تقلّل tolerance لـ 0 — floating-point legacy proportional يحتاج صبر

14. نموذج حساب المريض + قاعدة الجلسات المخطّطة (v1.1)

< أضيف في الإصدار 1.1 (commit cafd722). يكمل أقسام المحاسبة على مستوى العيادة (1-13) بتوثيق دفتر حساب المريض الفردي والقاعدة التي تربطه بصفحة المحاسبة.

14.1 الطبقتان

الطبقة	الصفحة	السؤال الذي تجيب عنه
حساب العيادة	accounting.html	كم حظّت العيادة وأنتجت وربحت في فترة؟ (cash-basis, أقسام 1-13)

الطبقة	الصفحة	السؤال الذي تجيب عنه
دفع المريض	+ patient-profile.html patients.html	كم على هذا المريض / كم دفع مقدماً؟

القاعدة الموحدة تضمن أن الطبقتين لا تتنافضان أبداً.

14.2 القاعدة الموحدة (Canonical Rule)

< ال split يُعدّ إيراداً مكتسباً (allocated) فقط إذا `is_earned = false` و مرتبط بجلسة مكتملة. غير ذلك (unearned)، أو على جلسة مخططة) = رصيد مقدّم متحرّك (unearned). < < رصيد المريض = (مجموع الجلسات المكتملة) - المخصّص. الجلسات المخططة تقدير منفصل، لا تدخل الرصيد أبداً.

مطابق OpenDental: دفعة الجلسة المخططة تبقى prepayment مخفية عن الرصيد والتقارير حتى تُعلّم مكتملة، وعندها يجري **Income Transfer** تلقائي فتصبح إيراداً.

14.3 computeFinancials() (في patient-profile.html)

```
// total += cost : planned → plannedTotal ، غير ذلك ، completedTotal
// لكل split:
var onCompleted = sp.session_id && sessStatus[sp.session_id] === 'completed';
if (sp.is_earned || !onCompleted) unearnedCredit += amount; // رصيد مقدّم
else
    allocatedToProduction += amount;
// splits (legacy): allocated = paid
var trueBalance = completedTotal - allocatedToProduction; // المنجز فقط
```

الحقول المُرجّعة: total (كل الجلسات، مرجعي) · completedTotal · plannedTotal · paid · allocatedToProduction · unearnedCredit · trueBalance.

14.4 FIFO — مكتمل فقط (buildFifoSplits)

```
// (legacy '' → completed) الأهلية: جلسات مكتملة فقط
return s && s.id && (s.status || 'completed') === 'completed' && (parseFloat(s.cost) || 0) > 0;
```

الفائض بعد تغطية الجلسات المكتملة → split بـ `is_earned=true`. لا يُخصّص أبداً لجلسة مخططة. عند تعليم جلسة مخططة كمكتملة، تُستدعى `reallocatePatientFunds()` فينتقل الرصيد المقدّم إليها تلقائياً (Income Transfer).

14.5 حالات الدفع الأربع (الرأس + سجل المرضى)

أولوية مع `epsilon = 0.5`:

الحالة	الشرط	اللون	العرض
متبقي	<code>trueBalance > 0</code>	أحمر	ل.س {tb} · متبقي
دفع مقدّم	<code>unearned > 0</code> (أو فائض قديم (tb < 0)	أزرق	ل.س {un} · دفع مقدّم
مسدّد	<code>completedTotal > 0</code> ≈ 0	أخضر	الرصيد والمسدّد
بلا حركة / —	لا علاج منجز ولا دفعات	رمادي	—

- **سجل المرضى (patients.html):** `loadFinancials()` تجلب 3 استعلامات batched (`ledger_sessions` + `ledger_payments` + `payment_splits`) عبر `in('patient_id', ids)`. وتطبّق نفس القاعدة الموحدة حرفياً → السجل يطابق صفحة الملف ١٠٠٪. لا تعتمد على الحقل المخزّن `patients.payment` للعرض.
- **عدّاد "مستحقات متبقية"** صار **مجموع** المبالغ المتبقية الحقيقي (لا عدّ بسيط).
- **بطاقة الرأس "المنجز"** `completedTotal`. والمخطّط يظهر في تبويب المدفوعات كبطاقة منفصلة "مخطّط (لم يُنجز)" مع ملاحظة "غير محتسب ضمن الرصيد".

14.6 معالجة البيانات القديمة (Self-Heal)

عند فتح ملف مريض، إذا وُجد split مكتسب (is_unearned=false) على جلسة مخطّطة (من سلوك FIFO القديم):

```
if (_needsHeal) { await reallocatePatientFunds(); await syncPaymentStatus(); }
```

يُعيد بناء ال splits مرة واحدة فتتحوّل تلك المبالغ إلى unearned حقيقية في DB. العرض صحيح فوراً عبر القاعدة الموحّدة حتى قبل التصحيح؛ وال self-heal يصحّ الصفوف المخزّنة فتصبح **صفحة المحاسبة** صحيحة أيضاً (لأنها تجمع حسب is_unearned).

14.7 السيناريوهات

#	الحالة	النتيجة
E1	جلسة مخطّطة فقط، بلا دفع	رصيد "-" + سطر تقدير المخطّط
E2	مخطّط + دفعة مقدّمة	دفع مقدم بالمبلغ
E3	منجز مدفوع بالكامل + مخطّط	مسدّد + تقدير منفصل
E4	منجز مدفوع جزئياً	متبقي على المنجز فقط
E5	منجز 100 مدفوع 150 + مخطّط 200	دفع مقدم 50 ← ثم عند إكمال المخطّط ينتقل الرصيد إليه تلقائياً
E6	بيانات قديمة (split على مخطّط)	self-heal يصحّحها عند الفتح

14.8 ملاحظات

- **index.html (لوحة التحكم): v1.2 — مُصلّحة.** صارت تحسب حالة الدفع لحظياً من الدفتر عبر computeDashFinancials + dashPayInfo بنفس القاعدة الموحّدة حرفياً (بطل الاعتماد على الحقل الثنائي المخزّن patients.payment). فمريض دافع مقدّم يظهر «دفع مقدّم» (أزرق) لا «متبقي»، وعدّاد «مستحقات» على اللوحة صار **مجموع** المتبقي الحقيقي. القاعدة الآن مطبّقة عبر **4 أسطر**: acc: counting.html + patient-profile.html + patients.html + index.html.
- **الخصم / الإعدام / الاسترداد: v1.2 — منقّذة** عبر account_adjustments (Migration 45). الاسترداد بالواجهة يُسجّل كتسوية ki nd='refund' بمبلغ موجب (لا تُدخّل دفعة سالبة يدوياً — حقل الدفع يبقى min=0). القسم 15 يشرح الدمج الكامل.
- **DELETE→INSERT في reallocatePatientFunds** غير ذرّي (لا backend). محمي بـ retry + re-entry guard واحد + toast عند الفشل. **تحسين مئاة مستقبلية:** تحويله لـ Postgres RPC ذرّي. (ما زال قائماً).

15. التسويات: خصم / إعدام / استرداد (v1.2 — Migration 45)

< أُضيف في الإصدار 1.2. سدّ أكبر فجوة وظيفية مقابل OpenDental والمنافسين العرب: القدرة على **تخفيض شحنة** (خصم/إعدام) أو **إعادة نقد** (استرداد) دون كسر invariants المحاسبة.

15.1 الجدول account_adjustments (Migration 45)

```
create table account_adjustments (
  id          uuid primary key default gen_random_uuid(),
  doctor_id   uuid not null references auth.users(id), -- legacy owner column
  patient_id  uuid references patients(id) on delete set null,
  kind        text not null check (kind in ('discount','write_off','refund')),
  amount      numeric not null check (amount > 0),      -- دائماً موجب؛ النوع يحدّد الإشارة
  date        date not null default current_date,
  note        text,
  created_at  timestampz default now()
);

-- RLS: (doctor_id = auth.uid()).
-- FK patient_id = ON DELETE SET NULL → clinic-level (تبقى) حذف المريض لا يفقد سجل التسوية.
```

ثلاثة مبادئ: (1) amount موجب دائماً والإشارة المحاسبية مشتقة من kind . (2) الحقل التاريخي date من نوع DATE → مقارنة مباشرة
 بلا توسيع timezone (خلاف payouts/labs). (3) graceful بالكامل — loadAdjustments على DB قبل Migration 45 يُرجع [] بصمت
 (isMigrationMissing).

15.2 دلالة الأنواع الثلاثة

النوع	kind	المعنى	الأثر النقدي	يؤثر على
خصم	discount	تخفيض السعر للمريض (حسم)	لا نقد يتحرّك	يقلّل صافي الإنتاج فقط
إعدام	write_off	شطب مبلغ غير قابل للتحصيل	لا نقد يتحرّك	يقلّل صافي الإنتاج فقط
استرداد	refund	إعادة نقد دفعه المريض	نقد يخرج	يقلّل صافي الإيراد + صافي الربح

< القاعدة الجوهرية: الخصم/الإعدام ليسا مصروفاً — هما تخفيض للشحنة لا تدقّق نقدي، فلا يلمسان الربح أبداً. الاسترداد نقد خارج فيقلّل الإيراد المحصّل والربح. هذا تمييز محاسبي حاسم: لا تستطيع «تحصيل» خصماً، ولا تُعدّ خصماً خسارة.

15.3 طبقة العيادة — computeAdjustmentTotals() (في accounting.html)

```
function computeAdjustmentTotals() {
  var chargeReductions = 0; // discount + write_off
  var refundsTotal = 0; // refund
  for (var i = 0; i < adjustments.length; i++) {
    var a = adjustments[i];
    var amt = Number(a.amount) || 0;
    if (amt <= 0) continue;
    if (a.kind === 'refund') refundsTotal += amt;
    else if (a.kind === 'discount' || a.kind === 'write_off') chargeReductions += amt;
  }
  return { chargeReductions, refundsTotal, adjTotal: chargeReductions + refundsTotal };
}
```

ثم في render() تُشتقّ الأرقام الصافية من الإجمالية:

```
var grossRevenue = summary.revenue; // المعادلة 1
var grossProduction = summary.production; // المعادلة 2

var netRevenue = grossRevenue - adj.refundsTotal; // الاسترداد نقد خارج
var netProduction = grossProduction - adj.chargeReductions; // الخصم تخفيض شحنة

var netProfit = netRevenue - totals.totalExpenses;
var profitMargin = (netRevenue > 0) ? (netProfit / netRevenue) * 100 : null;
var collectionRate = (netProduction > 0) ? (grossRevenue / netProduction) * 100 : null;
```

لماذا $\text{collectionRate} = \text{grossRevenue} \div \text{netProduction}$ ؟ لا تستطيع تحصيل مبلغ حسمته. لو شحنت 100k وحسمت 10k، فالمستحق الفعلي 90k؛ لو دفع المريض الك90 فمعدّل التحصيل $90k/90k = 100\%$ (لا 90%). استعمال الإنتاج الإجمالي في المقام يخس التحصيل لأنه يعدّ الك10 المحسومة كأنها لم تُحصّل.

15.4 لماذا التسويات clinic-level لا per-provider

التسويات تُخصّم مرّة واحدة على مستوى العيادة ولا تُنسب لطبيب. السبب معماري: اختبارات الهوية (Test A/B) تتحقّق من Conservation على الأرقام الإجمالية ($\sum \text{providerRows.collection} = \text{grossRevenue}$). لو وُزّعت التسويات على الأطباء لاحتاج كل bucket تعديلاً وتعقّد الإثبات. بإبقائها clinic-level:

- ✓ Test A/B/C/D/E تبقى خضراء (لا تتأثر بالتسويات إطلاقاً - GROSS تعمل على) تبقى خضراء
- ✓ الأرقام المافية مشتقة فوق طبقة الإثبات، فلا تكسرهما

< هذا قرار هندسي موثق في الكود: «they are NOT attributed per provider, so» These are CLINIC-LEVEL deductions shown separately — they are NOT attributed per provider, so»
 «the gross per-provider conservation (Test A/B) is untouched»

15.5 طبقة المريض — الدمج في computeFinancials() (في patient-profile.html)

```
// chargeReductions: تقلل الشحنة؛ refund: المستحق فتح يعيد فتح المستحق refund تقلل الشحنة؛
var chargeReductions = 0, refundsTotal = 0;
adjustments.forEach(function(a){
  var amt = parseFloat(a.amount) || 0;
  if (amt <= 0) return;
  if (a.kind === 'refund') refundsTotal += amt;
  else if (a.kind === 'discount' || a.kind === 'write_off') chargeReductions += amt;
});
var refundFromUnearned = Math.min(refundsTotal, unearnedCredit); // الاسترداد يستهلك الرصيد المقدم أولاً
var refundFromEarned = refundsTotal - refundFromUnearned; // الفائض يعيد فتح المستحق على المنجز
unearnedCredit = unearnedCredit - refundFromUnearned; // الرصيد الفعلي بعد الاسترداد

var netCompleted = completedTotal - chargeReductions; // الشحنة الفعلية بعد الحسومات
var trueBalance = netCompleted - allocatedToProduction + refundFromEarned;
```

منطق الاسترداد على مستوى المريض (الأهم): الاسترداد يأكل الرصيد المقدم (unearned) أولاً؛ فقط ما يتجاوز الرصيد المقدم (r) (efundFromEarned) يُعيد فتح مستحق على إنتاج منجز. هذا صحيح محاسبياً: إعادة دفعة مقدّمة مجردة لتقليص الالتزام، بينما إعادة مال مكتسب تخلق ديناً جديداً.

15.6 السيناريوهات (مؤكدة بتشغيل الكود الفعلي)

#	الحالة	النتيجة
A1	شحنة 100k، دفع 90k، خصم 10k	المستحق 0 · معدل التحصيل 100% (دفع كامل المستحق بعد الخصم)
A2	شحنة 100k، دفع 60k، خصم 20k	trueBalance=20k · netCompleted=80k (80-60)
A3	شحنة 100k، دفع بالكامل ثم خصم 20k	trueBalance=-20k → أصفر » يستحقه المريض 20k
A4	رصيد مقدّم 30k + استرداد 30k	الاسترداد يأكل الرصيد → unearned=0 · المستحق 0 «مسدّد»
A5	منجز مدفوع بالكامل + استرداد 30k	لا رصيد مقدّم → refundFromEarned=30k → trueBalance=30k «متبقى»
A6	عيادة: إيراد 90k، خصم 10k، استرداد 5k	netProduction=90k · netRevenue=85k · معدل التحصيل 100% · Test A-E خضراء على الإجمالي

15.7 العرض (UI)

- بطاقات الملخص (accounting.html): عند وجود تسويات تظهر سطور فرعية: «صافي بعد الخصم: {netProduction}» تحت الإنتاج، «صافي بعد الاسترداد: {netRevenue}» تحت الإيراد، ومقام معدل التحصيل يتبدّل إلى «الإيراد ÷ صافي الإنتاج».
- بطاقة «التسويات» (accounting.html): قسم مستقل (renderAdjustmentsSection) يظهر فقط عند وجود تسويات؛ يفصل إجمالي الخصم/الإعدام عن الاسترداد.
- بطاقة «التسويات» (patient-profile.html): مودال إضافة (نوع + مبلغ + ملاحظة) + قائمة + حذف، مع تحديث فوري للرصيد عبر القاعدة الموحدة.

• سجل المرضى (patients.html) + لوحة التحكم (index.html): تجلبان التسويات (refunds / chargeReductions) ضمن نفس الاستعلامات المجمعة فتعكسان الرصيد الصحيح فوراً.

15.8 ملاحظات للمطوّرين

- التسويات لا تُنشئ splits ولا تلمس payment_splits — هي طبقة خصمٍ منفصلة فوق الإيراد/الإنتاج المحسّوبين من ال splits.
- عند إضافة نوع تسوية جديد: حدّث ال CHECK في Migration، وفرعِي kind في كلتا الدالتين (compute + computeAdjustmentTotals) (Financials) — قاعدة #66 (نمط على مستوى الكود كاملاً).
- التسويات لا تدخل اختبارات الهوية (التي تعمل على GROSS عمداً). أيّ تحقّق من صحّة الأرقام الصافية يكون يدوياً عبر السيناريوهات أعلاه.

الخلاصة

نظام محاسبة SyDent مبني على:

- **Cash basis** (محاسبة نقدية) — مطابق المعيار الدولي + Dental CFO + OpenDental
 - **Conservation invariants** — كل ل.س مُسجّل في bucket واحد بالضبط
 - **Canonical earned-split rule** — الإيراد يُحسب على ال splits المكتسبة فقط (splitIsEarned)، مطبّقة عبر 4 أسطح قراءة + 3 كُتاب FIFO
 - **Adjustments** — خصم/إعدام/استرداد (Migration 45) clinic-level فوق طبقة الإثبات
 - **Timezone safety** — يعمل في أي local timezone بدون فقدان data
 - **Defense in depth** — layers 3 لحماية Owner-only access
 - **Live identity tests** — Ctrl+Shift+D للتحقق الفوري (على الأرقام الإجمالية)
- عند أي شك، افتح debug panel وتأكد أن كل tests خضراء ✓. لو أي failed → ابحث في ال bucket المعني (Revenue/Production /Expense/Lab).

النهاية

تاريخ آخر تحديث: 05 يونيو 2026 (الإصدار 1.2) يقابل commits: e521c25 → 12e7d22 → 1b8246b (إصدار 1.0) → 7dc65c9 → 8840674 → 4700e41 → a121844 → 7051e91 → d55f089 → 8707815 → caf722 (إصدار 1.1) → 0303eb7 → ebd5080 (إصدار 1.2، سياق v77) ال Author: د. أيهم غنيم + Claude (Anthropic) ab6efea

سجل تغييرات الإصدار 1.2 (05 يونيو 2026) — جلسة مراجعة المحاسبة

- **8707815** — إصلاح ال drift ال FIFO: قاعدة «الجلسات المخطّطة لا تستقبل دفعات» طبّقت على الكُتاب الثلاثة (patient-profile.html + buildFifoSplits + appointments.html _apptBuildFifoSplits + settings.html _backfillBuildFifo) — كانت في patient-profile فقط، فكانت النسختان الأخريان تخصّصان الرصيد المقدّم لجلسات مخطّطة كإيراد مكتسب (نفخ إيراد صامت).
- **8707815** (تتمة) — تحسين الإيراد في accounting.html : loadSplitSessionStatuses() + splitIsEarned() يطبّقان القاعدة الموحّدة لحظة القراءة (5 مواقع) فالملخّص صحيح بالبناء.
- **d55f089** — ميزة التسويات على مستوى المريض (Migration 45): جدول account_adjustments + دمج في computeFinancials + بطاقة «التسويات» + مزامنة patients.html.
- **7051e91** — rollup التسويات على مستوى العيادة في accounting.html (صافي الإيراد/الإنتاج، أسطر منفصلة، التسويات clinic-level لا per-provider).
- **8840674** — مزامنة لوحة index.html للرصيد الحيّ (dashPayInfo / computeDashFinancials : متبقي/دفع مقدّم/مسدّد + مستحقات حقيقية).
- **4700e41** — إصلاح صندوق مودال الجلسة المتسرّب (استعادة <div class="modal-overlay" id="sessionModal">).
- **a121844** — توحيد cache-bust فليت-وايد (لاحقاً وصل 20260601b30).
- **تحقّق v1.2**: تشغيل الكود الفعلي على بيئة معزولة — طبقة العيادة 61/61، طبقة المريض 26/26، كل اختبارات الهوية A-E خضراء. صيغة JS سليمة، توازن div مضبوط، صفر أسرار client-side.

سجل تغييرات الإصدار 1.1 (28 مايو 2026)

- **7dc65c9** — سجل المرضى: حالة الدفع تُحسب لحظياً من الدفتر + حالة "دفع مقدّم" جديدة (بدل الحفل المخزّن البايث).

- ebd5080 — القاعدة الموحّدة: الرصيد على المنجز فقط، والجلسات المخطّطة لا تكتسب إيراداً (FIFO مكتمل-فقط) + self-heal للبيانات القديمة.
- cafd722 / 0303eb7 — تنسيق: نقل ملاحظة "غير محتسب ضمن الرصيد" تحت بطاقة "مخطط (لم يُنجز)"، الرقم بالأحمر.